

AI RESUME SYSTEM

Complete Project Documentation

Free No API Streamlit Python

88

Job Roles

3

Modules

0₹

Total Cost

244

Unique Skills

Yash · Semester 6 · 2025

1. Project Overview

The AI Resume System is a full-stack web application built entirely with Python and Streamlit that helps students and job seekers build, analyze, and optimize their resumes — and then match them to real job roles using a dataset of 88 positions.

What makes this project stand out among student projects is that it achieves the look and feel of an AI-powered tool without using any paid APIs, external AI services, or cloud subscriptions. Everything runs locally on your machine after a simple pip install.



The Core Problem It Solves

Students don't know if their resume will pass ATS (Applicant Tracking Systems)
Most resume builders are either paid or don't show live previews
Job seekers don't know which skills they're missing for target roles
There's no free tool that combines building + screening + matching in one place

What it does in plain English

Module 1 — Resume Builder: You fill in a form. Your resume builds itself on the right side as you type. Hit one button to download it as a PDF. Hit another to auto-generate your professional summary.

Module 2 — AI Screening: You upload any resume file. The app extracts your skills, scores your resume from 0–100 across four dimensions, checks ATS compatibility, and gives you 5 specific things to fix.

Module 3 — Job Matching: You pick your skills or upload a resume. The app scans 88 real job roles, scores how well you match each one, shows you the skill gaps, and tells you exactly what to learn next.

2. Tech Stack & Dependencies

Library	Version	Purpose
streamlit	latest	Web UI framework — entire frontend, routing, state management
pandas	latest	Load and query the jobs.csv dataset
PyPDF2	latest	Extract text from uploaded PDF resumes
python-docx	latest	Extract text from uploaded DOCX resumes
reportlab	latest	Generate downloadable PDF resumes
re (built-in)	stdlib	Regex for email/phone/skill extraction from resume text
random (b-in)	stdlib	Pick from summary templates for the generator
os (built-in)	stdlib	Find jobs.csv across different paths
io (built-in)	stdlib	In-memory file handling for PDF generation

Why These Choices?

Streamlit was chosen because it lets you build a full web app in pure Python — no HTML, no JS, no CSS framework needed (though we do inject custom CSS for styling). It handles routing, state, file uploads, and rendering all in one.

ReportLab generates PDFs programmatically. It's the industry standard for Python PDF generation — fully offline, no API calls, complete control over layout.

PyPDF2 and python-docx handle the two most common resume formats. Combined they cover ~95% of resumes candidates submit.

3. File Structure & Architecture

PROJECT ROOT

```
my_project/
├── app.py                ← Entry point. Home screen with navigation.
├── shared_css.py         ← All CSS in one file, imported by every screen.
├── jobs.csv              ← Dataset: 88 job roles with required skills.
├── requirements.txt      ← pip dependencies.
└── pages/
    ├── screen1_resume_builder.py ← Module 1
    ├── screen2_ai_screening.py   ← Module 2
    └── screen3_job_recommendations.py ← Module 3
```

Why the pages/ folder?

Streamlit's multipage app system requires screen files to be inside a folder called pages/

The app.py at root becomes the home page automatically.
st.switch_page("pages/screen1_resume_builder.py") handles navigation between screens.
Each page file is independent — it has its own set_page_config, layout, and state.

shared_css.py — Why This Exists

Instead of copy-pasting 200 lines of CSS into every screen file, all styling lives in shared_css.py as a Python string variable called SHARED_CSS. Each screen does:

```
from shared_css import SHARED_CSS
st.markdown(SHARED_CSS, unsafe_allow_html=True)
```

This means: change one color in shared_css.py → all 4 screens update. This is the closest you can get to a CSS design system in Streamlit.

4. Module Deep Dives

Module 1 — Resume Builder (screen1_resume_builder.py)

WHAT IT DOES

A two-column layout: left side is a structured form with 7 sections. Right side is a live HTML preview that re-renders every time any field changes. At the bottom is a PDF download using ReportLab.

THE 7 INPUT SECTIONS

- Personal Information — Name, email, phone, city, LinkedIn, GitHub
- Education — Degree, college, graduation year, CGPA
- Professional Summary — Text area + AI-style auto-generate button
- Skills — Dropdown from 40 pre-defined skills + custom input + remove
- Work Experience — Company, role, duration, responsibilities (expandable)
- Projects — Title, description, tech stack (expandable)
- Certifications & Achievements — Name + type tag (expandable)

THE AI SUMMARY GENERATOR (RULE-BASED)

When you click "Auto-Generate Summary", Python picks one of 5 pre-written templates at random using `random.choice()`. It then fills in placeholders with your actual data:

```
template = "A results-driven {degree} graduate from {college}
           with strong expertise in {top3}..."

filled = template.format(
    degree = "B.E. Computer Engineering",
    college = "XYZ University",
    top3 = "Python, Machine Learning, Docker"
)
```

This produces text that reads like AI wrote it, because the templates are written in professional third-person resume style. The top 3 skills are the first 3 items the user added to their skills list.

LIVE PREVIEW

The preview is built by concatenating an HTML string in Python, then rendering it with `st.markdown(..., unsafe_allow_html=True)`. Every input change triggers a Streamlit rerun, which rebuilds the HTML string with the new values. Two themes are supported:

- ATS Clean — White background, black text, green section headers. Maximum readability for ATS systems.
- Modern Dark — Black background, white text, green accents. Visually striking for portfolio use.

PDF GENERATION FLOW

When the user clicks Download, Python runs `make_pdf()` which uses ReportLab's Platypus layout engine:

1. Creates an in-memory BytesIO buffer (no temp files on disk)
2. Defines paragraph styles (font, size, color, spacing) for each section type
3. Builds a story list — ordered list of Paragraph and HRFlowable objects
4. Calls `doc.build(story)` which writes the PDF into the buffer
5. Returns `buffer.getvalue()` as bytes, which Streamlit serves as a download

STATE MANAGEMENT

All resume data lives in Streamlit's `st.session_state` dictionary. This persists across reruns (each user interaction triggers a rerun). Keys like `full_name`, `skills`, `experiences` etc. are initialized with defaults at startup and updated by every widget.

Module 2 — AI Resume Screening (screen2_ai_screening.py)

TEXT EXTRACTION PIPELINE

The app accepts PDF, DOCX, and TXT files. The extraction pipeline:

- 6. User uploads file via `st.file_uploader()`
- 7. File bytes are read into a `BytesIO` buffer (not saved to disk)
- 8. Format detected from filename extension
- 9. `PyPDF2.PdfReader` or `docx.Document` processes the bytes
- 10. All text is joined into one long string for analysis

THE SCORING FORMULA — 4 DIMENSIONS

The overall score is a weighted average of four component scores:

Formatting Score (25%)	25%	
Skills Relevance Score (30%)	30%	
Experience Quality Score (25%)	25%	
Keyword Match Score (20%)	20%	

Formatting Score — How it's calculated

Starts at 40 base points, then adds 10 points for each section detected:

- Has a professional summary section? +10
- Has a skills section? +10
- Has contact info (email or phone detected)? +10
- Has external links (GitHub, LinkedIn, portfolio)? +10
- Has education section? +10
- Has certifications or awards? +10

Penalties: Resume under 150 words → -20 points (too short). Resume over 900 words → -10 points (too long for one page). Maximum is capped at 100.

Skills Relevance Score — How it's calculated

The app checks the resume text against a dictionary of 80+ known technical skills (python, docker, sql, tensorflow, etc.). Formula: $\min(100, 30 + \text{count_of_detected_skills} \times 5)$. So finding 14 skills gives $30 + 70 = 100$.

Experience Quality Score — How it's calculated

Scans for "quality keywords" — action verbs and impact indicators:

improved, reduced, built, developed, designed, achieved, increased, optimized, automated, deployed, implemented, managed, led, created, launched, delivered, analysed, researched, scaled, %, award, winner, rank, certification, published

Formula: $\min(100, 30 + \text{quality_keyword_count} \times 7)$. If no experience section is detected at all, -30 penalty.

Keyword Match Score — How it's calculated

If a Job Description is pasted in: compares word sets between JD and resume using set intersection. Formula: $\min(100, (\text{common_words} / \text{total_jd_words}) \times 200)$.

Without a JD: scans against 20 ATS-critical keywords (python, java, sql, machine learning, docker, git, agile, etc.). Formula: $\min(100, 30 + \text{ats_keyword_hits} \times 5)$.

ATS SCORE — SEPARATE FROM RESUME SCORE

The ATS score specifically measures how likely a resume is to pass through Applicant Tracking System filters. It starts at 40 and adds:

- +15 if a skills section exists (ATS scans for skills sections specifically)
- +10 if contact info is present
- +10 if experience OR projects section exists
- +10 if the resume is over 200 words (minimum length threshold)
- +15 if keyword match percent exceeds 40%

A score of 60+ = ATS Friendly. Under 60 = ATS Risk.

SUGGESTION ENGINE

Suggestions are generated by a conditional rule system, not AI. Each rule checks a specific condition and pushes a targeted suggestion string:

```
if not has_summary:
    suggs.append("Add a Professional Summary – first thing recruiters read.")
if quality_hits < 3:
    suggs.append("Add measurable achievements with numbers...")
if len(detected) < 5:
    suggs.append("List more technical skills explicitly...")
```

After rule-based suggestions are added, the remaining slots are filled from a shuffled pool of 14 general improvement tips. Final output is always exactly 5 suggestions.

Module 3 — Job Recommendations (screen3_job_recommendations.py)

THE DATASET — JOBS.CSV

Dataset Stats

88 total job roles
244 unique skills across all roles
Skills separated by | (pipe) delimiter
2 columns: role, skills

Role Categories

- ML/AI (Data Scientist, ML Engineer, AI Researcher)
- Web (Frontend, Backend, Full Stack, React, Vue...)
- Mobile (Android, iOS, Flutter, Kotlin, Swift)
- DevOps/Cloud (Docker, K8s, AWS, Azure, GCP)
- Security (Ethical Hacker, Pen Tester, SOC...)
- Management (Product Manager, Project Manager)
- Design (UI/UX, Figma, Wireframing)
- Specialized (Blockchain, IoT, Robotics, Bioinformatics)

THE MATCHING ALGORITHM

The matching is a pure skill overlap calculation — simple but effective:

```
for each job in jobs_df:
    required_skills = job["skills_list"]    # list from CSV
    user_skills_lower = [s.lower() for s in user_input]

    have = [s for s in required if s in user_skills_lower]
    miss = [s for s in required if s not in user_skills_lower]

    match_percent = round(len(have) / len(required) * 100)
```

Results are sorted by match_percent descending. A 100% match means the user has every skill that job requires. A 0% match means no skill overlap at all.

SKILL GAP ANALYSIS

The gap is computed from the top 10 matched jobs (not all 88). This makes it focused on roles the user is almost qualified for — the skills that would unlock the most opportunities:

```
counter = {}
for job in top_10_results:
    for skill in job["missing_skills"]:
        counter[skill] = counter.get(skill, 0) + 1

gap = sorted(counter, key=counter.get, reverse=True)[:8]
```

A skill appearing as missing in 8 of the top 10 jobs means learning it would unlock 8 more good matches. This is the skill to learn first.

ROLE SUGGESTIONS BY LEVEL

The app buckets matched jobs into experience levels based on match percentage:

- Beginner roles — jobs where you match 30–60% of required skills
- Intermediate roles — jobs where you match 60–85% of required skills
- Advanced roles — jobs where you match 85%+ of required skills

If a bucket is empty (e.g. no advanced matches), it falls back to showing the top results from the full sorted list so no section is ever blank.

5. CSS & UI Design System

Color Palette

#111111	Black	Primary backgrounds, buttons, dark cards
#6aef8a	Green	Accents, badges, Module 1 color, progress fills
#c4b5fd	Purple	Module 2 badge, gap analysis card, Module 3 accent
#f0f2ee	Off-White	Page background — warm, not harsh white
ffffff	White	Card backgrounds, input fields
#333333	Dark Gray	Primary body text on white backgrounds
#888888	Mid Gray	Placeholder text, secondary labels

Typography

- DM Sans — Primary font for all UI text, labels, body copy. Clean, modern, readable on screens.
- Space Mono — Monospace font for numbers (scores, percentages), skill tags, code-style chips. Creates visual contrast.
- Both loaded from Google Fonts via @import in the CSS.

The Placeholder Fix — Most Common Streamlit Styling Bug

The most invisible bug in Streamlit: placeholder text in input fields appears completely invisible on light backgrounds. This is because Streamlit sets input backgrounds to near-white (#fafafa) and the browser's default placeholder color is rgba(0,0,0,0.35) — nearly transparent on light colors.

The fix requires three separate CSS rules to cover all browsers:

```
/* Modern browsers */
input::placeholder, textarea::placeholder {
  color: #888888 !important;
  opacity: 1 !important;
}
/* Chrome, Safari, iOS Safari */
input::-webkit-input-placeholder { color: #888888 !important; opacity: 1 !important; }
/* Firefox */
input::-moz-placeholder { color: #888888 !important; opacity: 1 !important; }
```

Also critical: set background to pure #ffffff (not #fafafa) so the grey placeholder has sufficient contrast ratio.

Mobile Responsiveness

Streamlit doesn't natively stack columns on mobile. The fix is a CSS media query that overrides Streamlit's flex layout:

```
@media (max-width: 640px) {
  [data-testid="stHorizontalBlock"] {
    flex-direction: column !important;
  }
  [data-testid="stHorizontalBlock"] > div {
    width: 100% !important;
    min-width: 100% !important;
  }
}
```

```
flex: 1 1 100% !important;  
}  
}
```

Additional mobile tweaks: nav bar uses flex-wrap:wrap so the badge doesn't overflow. Hero title scales from 52px to 36px. Stats section wraps. Card padding reduces from 24px to 20px.

6. Data Flow Diagrams

Module 1 — Resume Builder Data Flow

Step-by-step data flow

1. User types in any text input field
2. Streamlit triggers a full page rerun (this is how Streamlit works)
3. The new value is saved to st.session_state (persists across reruns)
4. Python rebuilds the HTML preview string using current session_state values
5. st.markdown() renders the updated HTML in the right column
6. User clicks Download → make_pdf() reads from session_state → returns bytes
7. st.download_button() serves the bytes as a file to the browser

Module 2 — Screening Data Flow

Step-by-step data flow

1. User uploads a file → Streamlit holds it as an UploadedFile object
2. extract_text() reads bytes from the file object (no disk I/O)
3. Raw text string is passed to analyze(text, jd)
4. analyze() runs all rule-based scoring → returns a dict of results
5. Dict stored in st.session_state.result
6. Streamlit rerenders — right column reads from session_state.result
7. Score cards, progress bars, chip tags, suggestion items are built from the dict

Module 3 — Matching Data Flow

Step-by-step data flow

1. jobs.csv is loaded once at startup via @st.cache_data (cached, not reloaded on every rerun)
2. User uploads resume OR selects skills from multiselect
3. If upload: extract_text() → scan text against ALL_JOB_SKILLS list → store found skills
4. User clicks Find Matching Jobs → match_jobs(user_skills) runs
5. match_jobs() iterates all 88 rows, computes have/miss/percent for each
6. Results stored in st.session_state.job_results (sorted list of dicts)
7. compute_gap() finds most-missing skills across top 10 results
8. suggest_roles() buckets results by match % into beginner/mid/advanced
9. UI renders job cards, gap chips, learn items from session state

7. Setup & Running the Project

Prerequisites

- Python 3.8 or higher installed
- pip (comes with Python)
- A terminal / command prompt
- Any modern browser (Chrome, Firefox, Safari, Edge)

Step-by-Step Setup

STEP 1 — CREATE PROJECT FOLDER

```
mkdir ai_resume_system
cd ai_resume_system
```

STEP 2 — PLACE ALL FILES

Copy all downloaded files into the folder. Make sure jobs.csv is at the root level (same level as app.py), and the 3 screen files are inside a pages/ subfolder.

STEP 3 — INSTALL DEPENDENCIES (ONE TIME ONLY)

```
pip install streamlit PyPDF2 python-docx reportlab pandas
```

Or use the requirements file:

```
pip install -r requirements.txt
```

STEP 4 — RUN THE APP

```
streamlit run app.py
```

Browser opens automatically at <http://localhost:8501>. If it doesn't, open that URL manually.

Common Errors & Fixes

ModuleNotFoundError: streamlit → Run: pip install streamlit

ModuleNotFoundError: PyPDF2 → Run: pip install PyPDF2

FileNotFoundError: jobs.csv → jobs.csv must be in same folder as app.py (not inside pages/)

switch_page error → Screen files must be in a folder literally named pages/

Blank placeholder text → Make sure you copied shared_css.py to root folder

Port 8501 already in use → Run: streamlit run app.py --server.port 8502

Running Individual Screens

You can also run each screen standalone (without home navigation) for testing:

```
streamlit run pages/screen1_resume_builder.py
streamlit run pages/screen2_ai_screening.py
streamlit run pages/screen3_job_recommendations.py
```

8. Known Limitations & Scope

Limitation	Why / Workaround
Scanned PDF resumes fail	PyPDF2 can only extract text from text-based PDFs. Image-based scanned resumes return empty. Fix: use a text-based PDF or type in Module 1.
Session state not persistent	If you close the browser tab and reopen, all filled data is lost. Streamlit doesn't persist state by default. Fix: download resume before closing.
Skill matching is exact string	If resume says "ML" but dataset has "machine learning", it won't match. Case-insensitive but abbreviations can miss. Fix: add abbreviations to known skills list.
No real job listings	The 88 jobs are role templates from jobs.csv, not live scraped listings. There's no company name, salary, or apply link. For a real app, integrate a Jobs API.
Name extraction is heuristic	Name detection looks at the first 5 lines for a 2-5 word sequence of capitalized words. Can be wrong if resume has a header image or non-standard layout.
Single user only	Streamlit runs one session per browser tab. There's no multi-user login, authentication, or cloud storage. Data lives in browser memory only.

9. Possible Future Improvements

Short-term (Easy Wins)

- Add more resume templates (modern, creative, minimalist)
- Add a dark/light mode toggle across all screens
- Export resume as DOCX in addition to PDF
- Add more job roles to jobs.csv (currently 88, target 300+)
- Add skill synonym mapping (ML = machine learning)
- Let users save resume data to a JSON file and reload it

Long-term (Major Features)

- Integrate a free Jobs API (Adzuna, RemoteOK) for live listings
- Add a real AI API (Claude / GPT) for smarter suggestions
- Add user authentication with a database (SQLite or Supabase)
- Build a resume scoring history / progress tracker
- Add interview question suggestions based on skill gaps
- Deploy to Streamlit Cloud so it's accessible from any device

10. Interview / Viva Questions & Answers

Pro Tip

These are the exact questions professors and interviewers ask about projects like this. Read them all.

Q1: Why did you not use an AI API? Isn't it a limitation?

Answer: It's intentional, not a limitation. Using a paid API would mean the project only works with a credit card and fails when the API goes down or hits rate limits. The rule-based approach makes this project 100% offline, free, and deterministic. The scoring logic is fully explainable — every score point can be traced to a specific rule. That's actually better than a black-box LLM response for a resume tool.

Q2: How does the AI summary generator work?

Answer: It uses Python's `random.choice()` to select from 5 pre-written template strings. It then uses Python's `str.format()` method to fill in placeholders with the user's actual data — their degree, college, and top 3 skills. The templates are written in formal resume language, so the output reads naturally. This is template-based text generation, a legitimate NLP technique predating LLMs.

Q3: What is ATS and why does it matter?

Answer: ATS stands for Applicant Tracking System. It's software used by most companies (especially large ones) to automatically screen resumes before a human sees them. ATS systems parse resumes for keywords, section headers, and formatting. A resume that uses tables, images, or unusual fonts can score 0 in ATS even if the candidate is qualified. Our ATS score specifically checks for the elements ATS systems look for: skills section, contact info, keyword density, and word count.

Q4: What is `st.session_state` and why do you use it?

Answer: Every time a user interacts with a Streamlit widget, the entire Python script reruns from top to bottom. Without `session_state`, all variables would reset to their default values on every keystroke. `st.session_state` is a persistent dictionary that survives reruns within the same browser session. We store all resume form data (name, skills list, experiences list, etc.) in `session_state` so users don't lose their data between interactions.

Q5: What is ReportLab and how does PDF generation work?

Answer: ReportLab is a Python library for programmatic PDF creation. We use its Platypus module, which works like a document flow engine. You define paragraph styles (font, size, color, spacing), then build a "story" — an ordered list of Paragraph, Spacer, and HRFlowable objects. `doc.build(story)` renders this into a PDF. We write to a BytesIO buffer instead of disk, then serve the bytes directly as a download through Streamlit. No temp files, no disk writes.

Q6: How does the job matching algorithm work?

Answer: It's a set intersection algorithm. For each of the 88 job roles in `jobs.csv`, we compare the job's required skills list against the user's skills list. The match percentage is: $(\text{number of matching skills} / \text{total required skills}) \times 100$. All 88 results are sorted descending by this score. The skill gap is computed from the union of missing skills in the top 10 results, weighted by frequency — a skill missing from 8 of 10 top jobs is the most important to learn.

Q7: Can this handle scanned resumes?

Answer: No. PyPDF2 can only extract text from digitally-created PDFs where the text is stored as actual characters. Scanned PDFs are images of text — they require OCR (Optical Character Recognition, e.g. Tesseract) to extract text. That's a future improvement — adding pytesseract as a fallback when PyPDF2 returns empty text.

Q8: What does @st.cache_data do in screen 3?

Answer: @st.cache_data is a Streamlit decorator that memoizes function results. Since every user interaction reruns the whole script, without caching, load_jobs() would read and parse jobs.csv on every single rerun — potentially hundreds of times per session. With @st.cache_data, the CSV is loaded once, the result is cached in memory, and subsequent calls return the cached DataFrame instantly. It's a critical performance optimization.

11. Glossary of Terms

ATS	Applicant Tracking System — software companies use to auto-filter resumes before human review
Streamlit	Python framework for building web apps without HTML/CSS/JS knowledge
session_state	Streamlit's persistent key-value store that survives page reruns within a browser session
ReportLab	Python library for creating PDF documents programmatically
PyPDF2	Python library for reading and extracting text from PDF files
python-docx	Python library for reading/writing Microsoft Word (.docx) files
BytesIO	Python's in-memory binary stream — lets us create files without writing to disk
@st.cache_data	Streamlit decorator that caches a function's return value to avoid re-running on every rerun
Platypus	ReportLab's high-level page layout engine used for PDF generation
Regex	Regular Expressions — pattern matching language used to extract emails, phones, skills from text
Set intersection	Mathematical operation: finds common elements between two sets. Used in job matching.
Rule-based AI	Intelligence built from explicit human-written rules and conditions, no machine learning involved
DM Sans	Google Font used for UI body text — clean, modern, highly legible on screens
Space Mono	Google Font used for numbers and code-style elements — monospace, technical feel
Pipe delimiter	The character used in jobs.csv to separate multiple skills: "python sql docker"